LogicFly: Joint Task–Viewpoint Planning for Long-Horizon Language-Guided Aerial Navigation in Mapped Environments

Anonymous Author(s)

***Abstract*—Long-horizon aerial inspection requires a robot to visit multiple locations, respect task dependencies, and revise its actions when expected objects are missing. In repeatedly accessed buildings, prior exploration provides reusable semantic maps for subsequent navigation requests. However, semantic goal grounding alone does not determine a feasible visit order, informative camera poses, or an appropriate response to negative observations. We present LogicFly, a language-guided aerial navigation framework that couples these decisions through a Conditional Partial-Order Task Graph (CPTG) and Joint Task–Viewpoint Planning (JTVP). CPTG separates mandatory precedence from observation-triggered branches and distinguishes the mapped search anchor from the verification target. JTVP jointly selects task order and three-dimensional observation poses within a rolling horizon using collision-aware travel costs and geometric visibility. An observation-driven execution policy retries local views before expanding the search and updates regional confidence from newly observed coverage. Habitat and physical UAV experiments demonstrate multi-room navigation, online target recognition, conditional task activation, local viewpoint retries, and cross-room recovery search.**

***Note to Practitioners*—This work is motivated by repeated visual inspection and inventory tasks in buildings that can be mapped before operation. An operator may request several checks, specify their required order, and prescribe an alternative when an expected item is missing. LogicFly represents these requirements explicitly and selects observation positions together with the visiting sequence. It also distinguishes an unsuccessful camera view from an exhausted search location, allowing another observation before a fallback is triggered. The approach is applicable to facilities with a reusable semantic map, reliable localization, and calibrated visual sensing. Physical UAV experiments demonstrate online target recognition, conditional task execution, and recovery across rooms. Practical performance depends on the quality of the prior map, localization, and visual observations. The separation of task logic from observation geometry allows inspection requests to reuse the same spatial representation while adapting the selected views to each task.**

***Index Terms*—Aerial robotics, task and motion planning, vision-and-language navigation, semantic mapping, visual verification.**

I. Introduction

Indoor aerial robots can support repeated visual inspection, inventory checks, and object search across rooms and floors. These applications require more than reaching an isolated destination: a request may specify several observations, require one check before another, and prescribe a fallback if an expected object is absent. Long-horizon navigation therefore involves deciding what to inspect, where to observe it, and when new evidence should change the remaining mission. Efficient coordination matters because unnecessary travel consumes flight time and delays completion of the overall request.

Many recurring deployments allow the environment to be surveyed before individual tasks arrive. Homes, offices, and managed facilities are repeatedly accessible, and their large-scale geometry often changes more slowly than the objects of interest. Reusing a semantic map is consequently a practical operating assumption for this setting. Visual-language maps and hierarchical scene graphs already demonstrate how prior observations can support language-grounded navigation [1], [2]. The central question is how to turn that prior knowledge into efficient, observation-driven execution when instructions contain multiple interacting requirements and the stored semantics may be incomplete.

Three gaps motivate our formulation. *First, a destination or fixed subgoal list does not fully express task logic.* Conventional VLN benchmarks established instruction following and continuous navigation [3], [4], and long-horizon VLN extends these settings to sustained multi-stage execution [5]. For inspection requests, however, mandatory precedence, freely reorderable checks, and conditional alternatives must be distinguished: imposing an unnecessary order wastes travel, whereas ignoring a required order changes the task. *Second, semantic task grounding does not uniquely determine an observation pose.* Scene-graph planners connect language to spatial goals [6], [7], but an aerial robot can inspect the same region from different heights and directions. Selecting a pose independently for each task can make the combined route unnecessarily long, and proximity to an object does not ensure visibility. *Third, negative evidence has different meanings at different scales.* Missing an object in one image may justify another view, while exhausting a location may justify searching elsewhere. Treating every missed detection as task failure, or treating every fallback as unrestricted search, loses this distinction.

We address these issues with **LogicFly**, which separates reusable map construction from task execution. The first stage organizes measured geometry and semantic observations into floors, rooms, and object instances. The second stage represents the request as a Conditional Partial-Order Task Graph (CPTG), generates task-conditioned observation poses, and applies Joint Task–Viewpoint Planning (JTVP) to a bounded set of currently executable tasks. Following each observation, an explicit recovery policy determines whether to retry locally, search another region, or resolve the task and evaluate its conditional rules. This design connects logical progress to

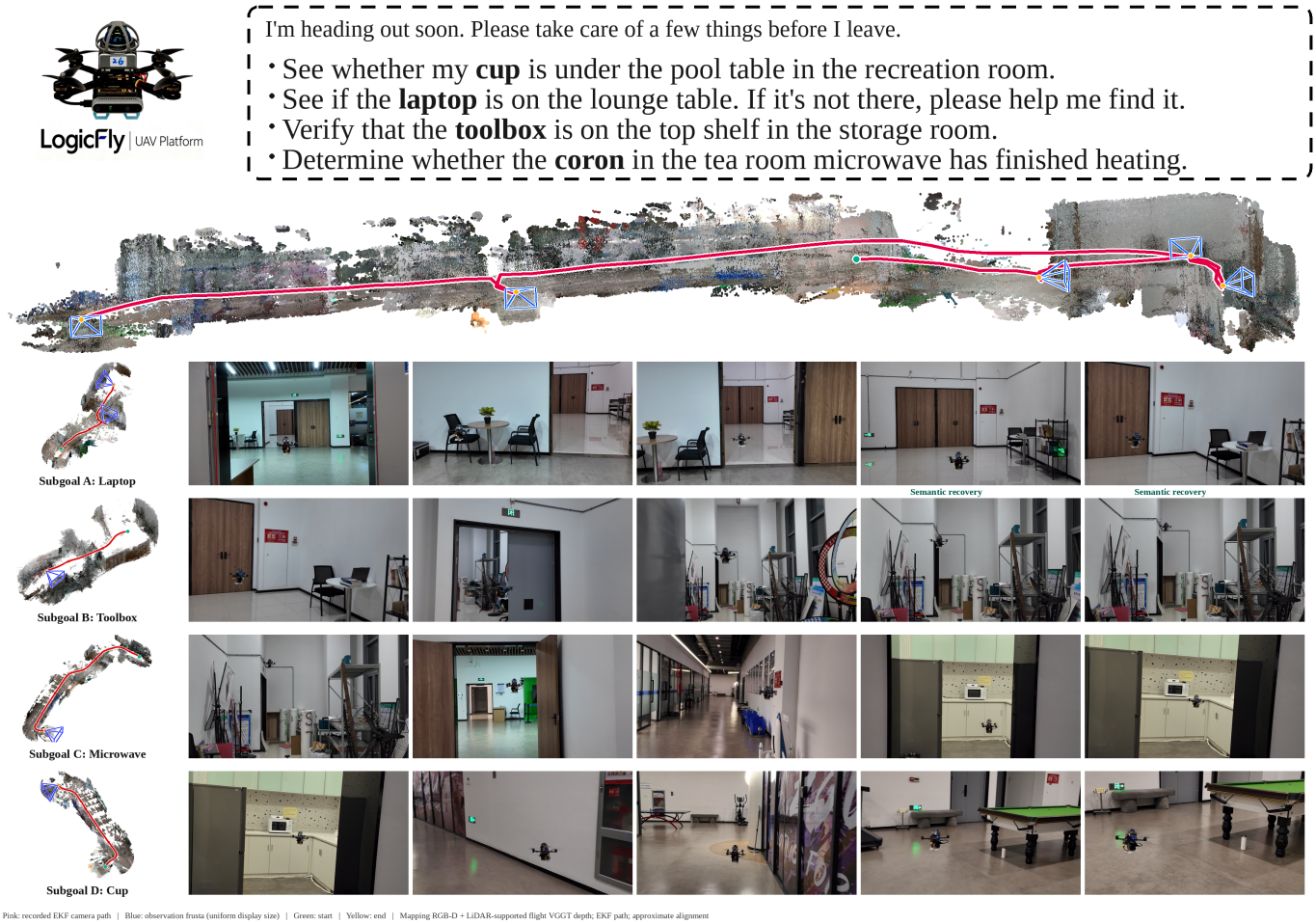

Fig. 1. Real-world execution of a single multi-task language instruction by LogicFly. The global reconstruction shows the continuous recorded camera trajectory and observation frusta. Each row pairs a local reconstruction with third-person views of laptop search and recovery, toolbox verification, microwave inspection, or cup verification. Reconstructions fuse VGGT depth with prior mapping RGB-D and LiDAR support; trajectories use onboard EKF odometry with approximate video alignment.

geometric observation opportunities without committing the whole mission to an order before its outcomes are known.

Our contributions are:

- A conditional partial-order execution model that separates precedence, outcome-triggered branches, and within-task recovery, while retaining distinct search anchors and verification targets.
- A rolling joint planner that couples task ordering with three-dimensional observation-pose selection, using finite candidate sets, visibility scores, and collision-checked travel costs.
- An evidence-driven execution mechanism that distinguishes local misses from terminal outcomes and updates regional search confidence using newly observed coverage.

Qualitative Habitat examples illustrate multi-target execution, recovery after negative observations, and conditional task activation. Physical UAV experiments demonstrate these closed-loop capabilities through online target recognition, conditional task execution, local viewpoint retries, and cross-room recovery search. Section V presents the experimental settings and examples.

II. Related Work

A. From Instruction Following to Persistent Spatial Memory

R2R established visually grounded route following [3], while VLN-CE moved execution from a predefined navigation graph into continuous environments [4]. These developments improve the connection between language and motion, but a single episode need not capture the repeated use of a building. Iterative VLN addresses experience reuse across successive instructions [8], and LH-VLN emphasizes consistency across extended subtask sequences [5]. Together, they motivate persistent memory and longer planning horizons. Our setting additionally requires explicit freedom among unordered tasks and explicit rules for activating alternatives after observations.

VLMaps grounds open-vocabulary and spatial language directly in a reconstructed map [1]. ConceptGraphs organizes observations into object-centric 3D scene graphs [9], and HOV-SG adds floor–room–object hierarchy and cross-floor navigation [2]. This progression makes large scenes easier to query, but a grounded object or region still leaves the robot's camera pose and the order of multiple inspections to be chosen. LogicFly builds on this semantic hierarchy to jointly select observation poses and the order of multiple tasks.

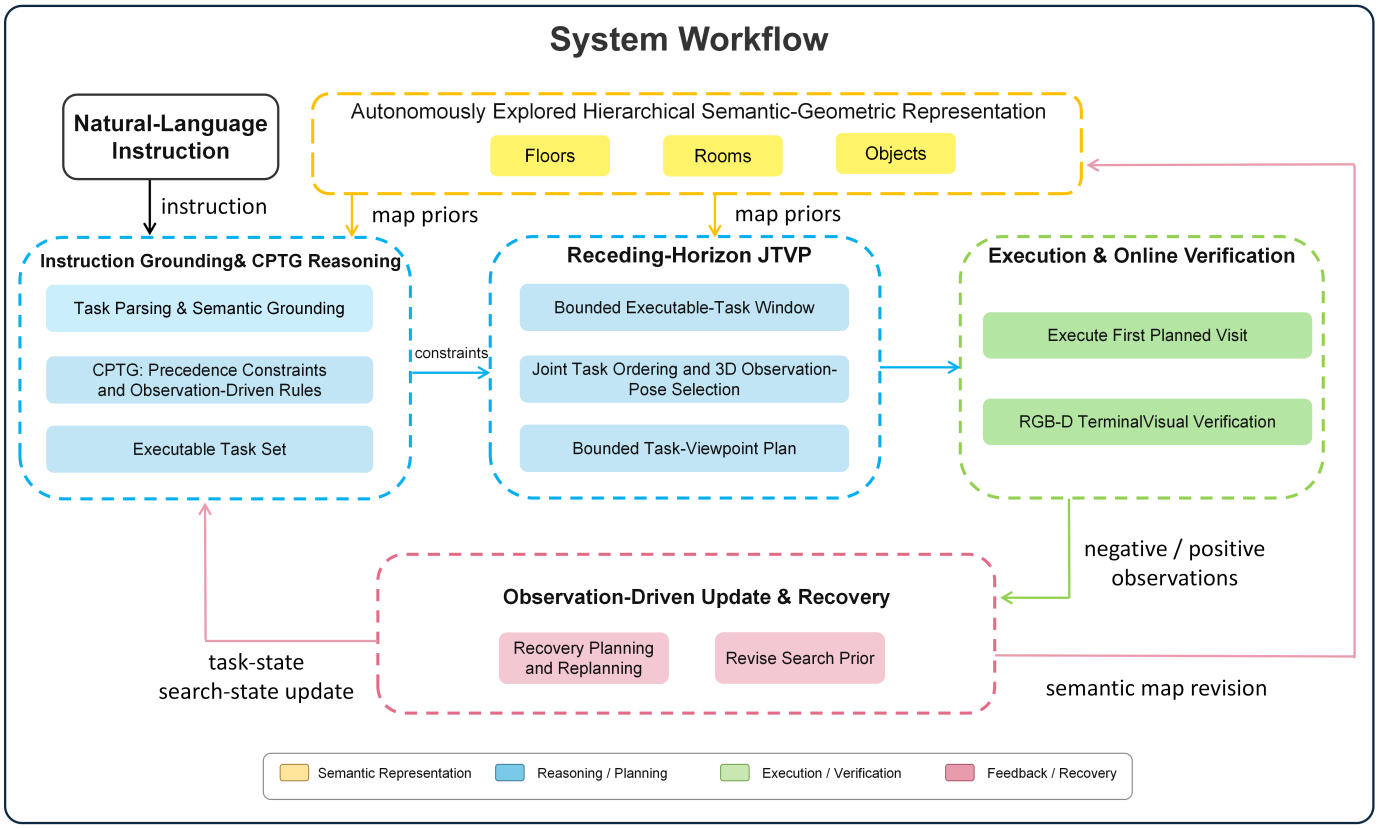

Fig. 2. LogicFly pipeline. Exploration and semantic mapping provide reusable spatial priors. A language request is represented as a CPTG; task-conditioned views and collision-aware costs support rolling JTVP. Observations update task and search states before replanning. The physical deployment is described in Section V-F.

B. Structured Task Planning and Observation Geometry

SayPlan combines hierarchical scene-graph search with classical planning and feasibility feedback, enabling language-conditioned plans in large multi-room environments [6]. OSGP gives temporal constraints a formal representation through an LTL automaton and performs optimal hierarchical search over a metric–semantic graph [7]. These works establish that structured task reasoning can be combined with geometric costs. Our distinction is the finite planning state: each task can have several admissible camera poses, and the planner chooses the task sequence and those poses together. This formulation couples task sequencing with observation geometry within a rolling planning window.

Continuous trajectory optimization addresses a complementary problem. MINCO provides a compact trajectory representation for smooth multicopter motion under geometric and dynamic constraints [10]. AirHunt further connects VLM semantics to continuous aerial object-navigation planning [11]. These developments motivate separating continuous trajectory optimization from discrete task planning. LogicFly concentrates on coordinating multiple logically related observation tasks; its discrete task–viewpoint objective uses a geometric travel estimate, and trajectory optimization follows the selected visits.

C. Language-Based Decisions and Closed-Loop Feedback

NavGPT uses language-model reasoning over observations and action history [12]. MapGPT supplements local observations with an online topological map and adaptive multi-step planning [13]; SpatialNav uses a scene graph acquired before task execution to provide global spatial context [14]. VLN-Zero similarly separates exploration from deployment and reuses cached task-location trajectories [15]. Building on prior exploration and structured memory, LogicFly coordinates observation tasks with precedence constraints and outcome-dependent continuations.

Inner Monologue demonstrates that success and environment feedback can revise language-based plans [16]. For the present observation tasks, the remaining issue is how to interpret a failed check without changing the meaning of the instruction. LogicFly assigns explicit semantics to a failed pose, an exhausted search location, and a terminal task outcome. This supports bounded autonomous recovery while keeping a user-specified conditional branch separate from a new search hypothesis.

III. System Overview and Problem Formulation

A. Operating Setting and System Pipeline

We consider one UAV performing navigation and visual observation in a previously surveyed indoor environment. The input is a natural-language request I, the current pose

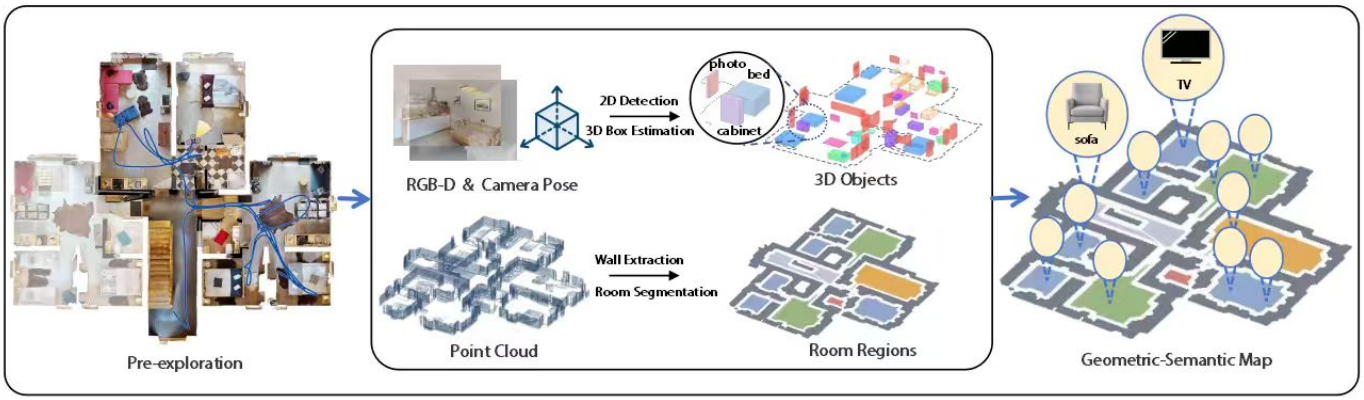

Fig. 3. First-stage map construction. Autonomous exploration provides RGB-D observations, camera poses, and point-cloud geometry. Object detection and 3D box estimation produce object instances, while wall extraction and room segmentation produce spatial regions. These outputs are integrated into a reusable semantic–geometric map.

$q_0 = (x, y, z, \psi)$, and a map $\mathcal{M} = (\mathcal{M}_g, \mathcal{M}_s)$. The geometric map $\mathcal{M}_g$ distinguishes occupied, observed-free, and unknown space. The semantic map $\mathcal{M}_s$ associates floors and rooms with object instances, geometry, and confidence. The camera has a fixed, level pitch; position and yaw determine the observation pose. The objective is to complete navigation and visual observation tasks while respecting their logical and geometric constraints.

Figure 2 summarizes two stages. During map construction, exploration supplies posed RGB-D observations and measured geometry; semantic fusion and spatial partitioning produce the reusable map. During execution, the instruction is represented as a CPTG whose grounded tasks define possible camera poses. JTVP chooses the next visit using a short lookahead. The executor moves to the selected pose, acquires evidence, and updates task and search states before the next planning step. The occupancy representation determines geometric feasibility, while room labels provide semantic context.

B. Planning State and Objective

The task representation is $\mathcal{G}_T = (\mathcal{V}, \mathcal{E}_{\text{prec}}, \mathcal{R}_{\text{cond}})$, where the three terms denote tasks, mandatory precedence edges, and conditional activation/skip rules. At decision time t, the execution state comprises the robot pose q_t, active tasks, resolved outcomes, attempted viewpoints, and regional search confidence. A task may be resolved with a negative outcome; resolution and successful target discovery are therefore different events.

For each task i, let $\mathcal{Q}_i$ contain its feasible observation poses. The planning decision selects a sequence of task–pose pairs from the currently executable tasks, minimizing a combination of travel length, a motion-time proxy, and terminal observation cost. Precedence, branch activation, occupancy, and visibility restrict these choices. The rolling policy commits the first selected visit and reconsiders subsequent visits after new evidence. Optimization is performed over a finite task window and candidate set.

IV. Methodology

The method connects four operations: constructing reusable spatial priors, representing task logic, jointly selecting tasks and viewpoints, and updating the plan from observations. The semantic map grounds candidate search locations, while the CPTG constrains which tasks can be executed. JTVP then chooses the next observation, and its outcome determines whether to retry, expand the search, or advance the task graph.

A. Reusable Semantic–Geometric Map

The mapping stage supplies spatial priors for later instructions. FALCON [17] provides autonomous exploration for collecting RGB-D observations, camera poses, and occupancy evidence. All observations are expressed in a common global frame. The geometric layer retains unknown space as distinct from observed free space, while the semantic layer stores floors, rooms, object labels, 3D boxes, and confidence. Figure 3 shows the construction pipeline.

1) Object and room semantics: Text-prompted SAM 3.1, from the SAM 3 model family [18], produces object boxes and masks from mapping images. The masks suppress background depth before Boxer [19] estimates 3D boxes. Multi-view fusion consolidates these observations into object instances, which are associated with spatial room regions by their centers. Room views and associated objects provide evidence for room labels; regions with insufficient evidence remain unknown. The resulting hierarchy supplies anchors for later search, including requests for objects that were not observed during mapping.

2) Wall-guided room partitioning: After exploration, wall-guided partitioning organizes each floor into room regions (Fig. 4). Extracted walls constrain a candidate floor domain. Spatial support is established by near-horizontal surfaces, free-space observations at multiple heights, multi-frame depth rays, or the recorded trajectory. Partitioning is restricted to supported cells, preventing isolated wall measurements from creating spurious room interiors. Unsupported cells retain their unknown status.

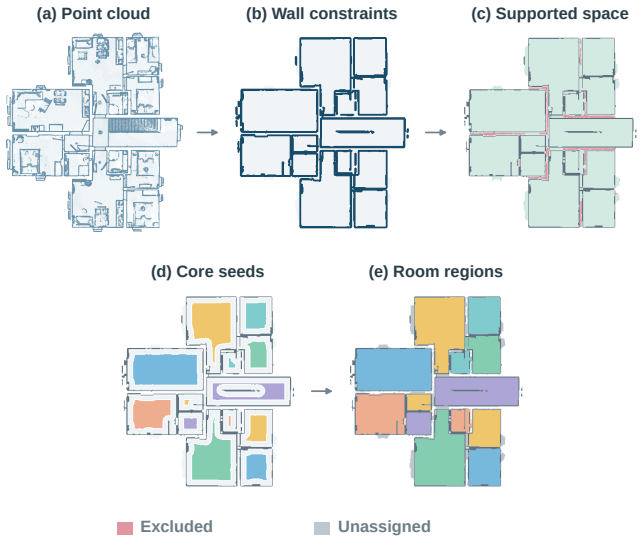

Fig. 4. Wall-guided room partitioning on a recorded second-floor map. (a) Measured point cloud. (b) Extracted walls constrain the candidate domain. (c) Independent observations retain supported space; rose cells are excluded. (d) Connected distance-field cores initialize seeds. (e) Masked watershed propagation and geometric merge checks yield region labels. Seed and region colors correspond; gray denotes unassigned supported space. All panels share the complete floor extent, scale, and orientation.

A distance transform identifies connected interior cores in the supported domain. Cores with sufficient area initialize a masked watershed, and geometric merge checks combine weakly separated regions. Supported components without a seed remain unassigned. Room labels organize semantic reasoning, while the 3D occupancy map independently determines flight feasibility, including obstacles within each room.

B. Conditional Partial-Order Task Graph

A language model converts the instruction into a CPTG (Fig. 5). Each node specifies an action, a mapped anchor, a verification target, spatial scope, and search intent. Directed precedence edges encode mandatory order; conditional rules associate an observed outcome with the activation or skipping of other tasks. Task anchors are grounded in the semantic map, and precedence constraints form a directed acyclic graph. Tasks governed by an activation rule remain inactive until its condition is met.

1) Anchor, target, and intent: The *anchor* defines where to inspect; the *verification target* defines what to detect. For "look for a bag under the table," the mapped table can anchor the below-table region even when no bag is stored in the map. A relation to another object can disambiguate the intended anchor without replacing it. This separation also determines the meaning of negative evidence: failure to detect the bag does not invalidate the table.

Task intent distinguishes a presence check from a locate-target request. A presence check accepts either discovery or a terminal negative result, whereas a locate-target request requires discovery or a successful explicit fallback. A generic request to keep searching retains the same task and permits autonomous recovery. An explicitly named fallback is represented as a conditional task. These policies are mutually exclusive for a source task, preserving the distinction between an operator's prescribed alternative and an inferred search location.

2) Eligibility and outcomes: Let $\mathcal{A}_t$ be the active pending tasks and $\mathcal{R}_t$ the tasks resolved with an outcome. The currently executable set is

$$\mathcal{E}_t = \{i \in \mathcal{A}_t : \text{pred}(i) \subseteq \mathcal{R}_t\}. \tag{1}$$

A resolved not-found outcome can satisfy a predecessor requirement while triggering a different branch. When a source task terminates, matching rules activate their designated tasks and apply skip rules; inactive alternatives associated with nonmatching rules are skipped. A failed intermediate view does not resolve the source task while its recovery policy still permits another attempt. This preserves the distinction between learning that one camera pose was unproductive and concluding that a conditional fallback should run.

Spatial relations constrain anchor grounding and observation-region construction; observations at the selected pose provide evidence of target presence.

C. Joint Task–Viewpoint Planning

JTVP converts each executable task into a finite set of camera poses and jointly chooses the visit order and one pose per admitted task. The generation and selection procedures preserve alternatives needed for local recovery while limiting the size of the global discrete optimization.

1) Task-conditioned candidate generation: An anchor and its relation define an observation region, represented by samples $\mathcal{P}_i = \{s_u\}$ on an object, a support surface, a surrounding neighborhood, or a below-object region. Candidate horizontal positions are sampled over azimuths and stand-off distances. Distances account for region geometry, requested observation detail, and horizontal field of view (FoV), subject to any explicit distance constraints. Candidate yaw faces the region center.

For a sampled horizontal position $\boldsymbol{x}$, let $\rho_u = \|\boldsymbol{x} - s_{u,xy}\|_2$ and let α_v denote vertical FoV. For a level camera, vertical inclusion of every region sample gives the height interval

$$\begin{aligned} h_u &= \rho_u \tan(\alpha_v/2), \\ \mathcal{H}_i(\boldsymbol{x}) &= \mathcal{H}_{\text{flight}} \cap \bigcap_{s_u \in \mathcal{P}_i} [s_{u,z} - h_u,\ s_{u,z} + h_u]. \end{aligned} \tag{2}$$

Here, $\mathcal{H}_{\text{flight}}$ denotes admissible flight heights. An empty intersection rejects the horizontal sample. Otherwise, the nominal height is chosen closest to the mean region height within the feasible interval. Additional heights can provide finer observation detail; task-specified heights remain subject to geometric and visibility constraints. Equation 2 controls vertical inclusion before occlusion is considered.

For pose $q = (x, y, z, \psi)$, let $\mathcal{V}_i(q)$ be the region samples inside both FoV limits and with a valid line of sight. The visibility score is

$$Q_i(q) = \frac{|\mathcal{V}_i(q)|}{\max(1, |\mathcal{P}_i|)}. \tag{3}$$

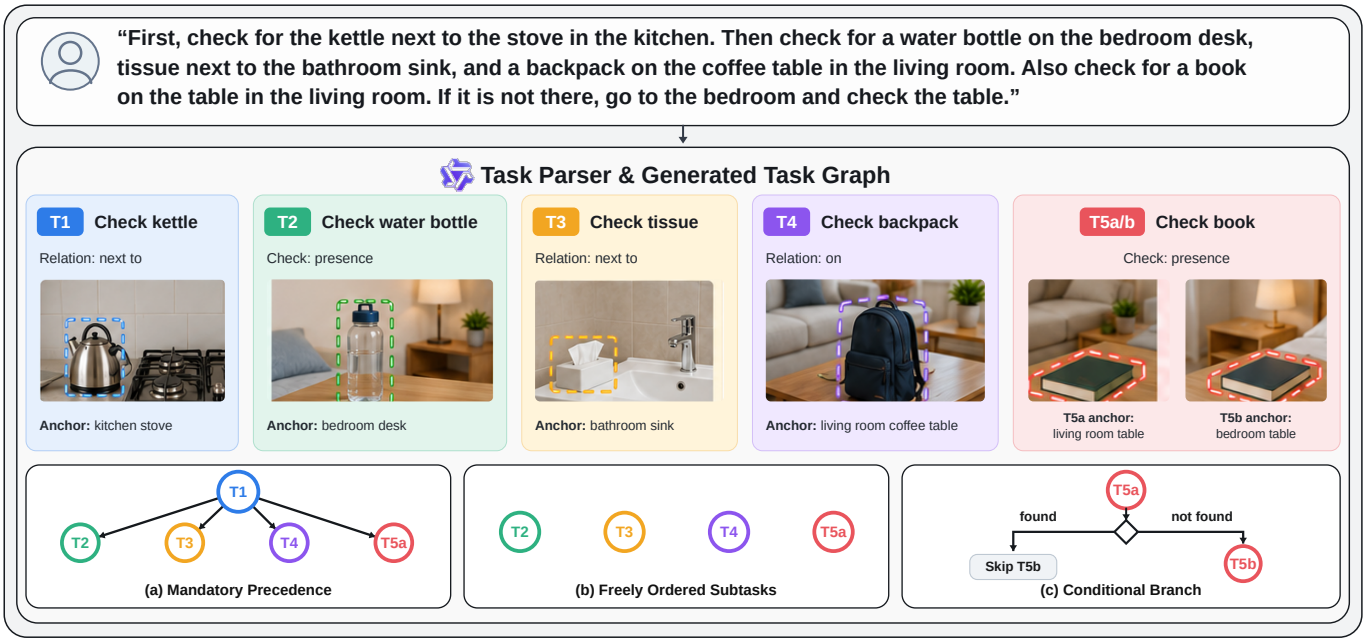

Fig. 5. Language instruction and its CPTG representation. Cards specify verification targets and mapped anchors; relations define search geometry. (a) T1 must resolve before T2–T4 and T5a. (b) These eligible checks have no mutual precedence. (c) A terminal not-found outcome at the living-room table (T5a) activates the bedroom-table check (T5b); a found outcome skips T5b. Scene images are illustrative.

The generator rejects invalid 3D states and disallowed directional or between-object configurations. For tasks requiring visibility, feasibility is established by visible samples in the observation region, allowing the camera to inspect a region around an occluded anchor center.

Feasible candidates receive a terminal cost

$$C_i^{\text{view}}(q) = \beta_d \frac{|d - d_i^{\text{pref}}|}{\ell_0} + \beta_Q (1 - Q_i(q)) + \beta_z \frac{|z - \bar{z}_i|}{\ell_0} + \beta_r \frac{d_i^{\text{ref}}}{\ell_0} + \beta_b \frac{d_i^{\text{between}}}{\ell_0}, \tag{4}$$

where d is stand-off distance, d_i^{pref} is its preferred value, $\bar{z}_i$ is mean sample height, d_i^{ref} is the mapped anchor's horizontal distance to the nearest reference, and d_i^{between} is the between-reference geometric penalty. The nonnegative weights β_d, β_Q, β_z, β_r, and β_b balance these terms. Inapplicable terms vanish, and ℓ_0 is a reference length that makes distance penalties dimensionless. The reference-distance term distinguishes anchor hypotheses, while the pose-dependent terms distinguish views of each anchor.

Each physical location hypothesis contributes a compact set of complementary views. Below-object regions first retain the most visible, lowest-cost pose at each feasible azimuth, then maximize angular diversity if pruning is needed. Other regions retain feasible height diversity and favor spatially distinct views with complementary visible samples. This yields $\mathcal{Q}_i$, with a bounded number of poses per location. Figure 6 shows the below-object case.

2) Transition cost: A* estimates collision-free travel through observed free space between candidate poses. For a returned path $P_t(q_a, q_b) = \{p_u\}$ and $\Delta\psi_{ab} = \text{wrap}(\psi_b - \psi_a)$, define

$$L_t(q_a, q_b) = \sum_u \|p_{u+1} - p_u\|_2,$$
$$\widehat{T}_t(q_a, q_b) = \max\left\{ \frac{L_t(q_a, q_b)}{v}, \frac{|z_b - z_a|}{v_z}, \frac{|\Delta\psi_{ab}|}{\omega_\psi} \right\}. \tag{5}$$

Here, v, v_z, and ω_ψ are nominal translational, vertical, and yaw rates. The maximum gives a travel-time proxy for discrete planning. An unreachable transition has infinite cost. For a destination pose belonging to task i, the transition score is

$$c_t(q_a, q_b) = \frac{L_t(q_a, q_b)}{\ell_0} + \eta_T \frac{\widehat{T}_t(q_a, q_b)}{\tau_0} + \eta_V C_i^{\text{view}}(q_b), \tag{6}$$

where τ_0 is a reference duration for time normalization, and η_T and η_V balance travel time and observation quality.

3) Task window and representatives: To bound planning cost, executable tasks are ranked by their best Euclidean-distance and terminal-cost estimate, and at most H tasks enter the window $\mathcal{W}_t$. Views at each physical location are ranked using distance, estimated motion time, and terminal cost. The initial optimization uses the highest-ranked representative at each location. If the complete task window is infeasible, additional representatives are progressively admitted, up to the full candidate set. This adaptive expansion preserves alternatives for feasibility while controlling the number of geometric path queries. Multiple anchor hypotheses contribute separate representatives.

4) Subset dynamic program: Let $\widetilde{\mathcal{Q}}_{i,t}$ be the admitted poses and $D(S, i, k)$ the minimum cost of visiting task subset S and ending at q_i^k. Initialization and transitions are

$$D(\{i\}, i, k) = c_t(q_t, q_i^k),$$
$$D(S \cup \{j\}, j, \ell) = \min_{i \in S, k} \left[D(S, i, k) + c_t(q_i^k, q_j^\ell) \right]. \tag{7}$$

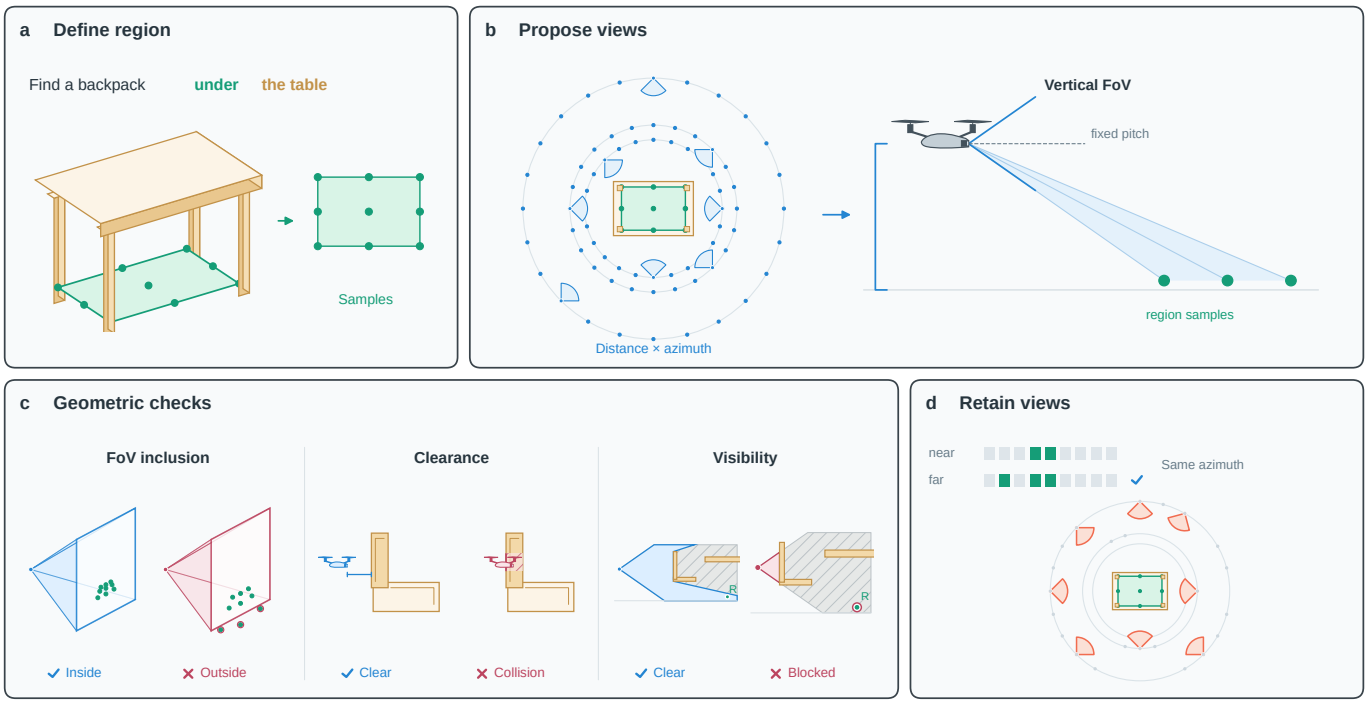

Fig. 6. Task-conditioned candidate viewpoint generation. (a) The mapped anchor and spatial relation define the observation region. (b) Positions are sampled over distances and azimuths, with yaw facing the region; at fixed camera pitch, the vertical field of view constrains height. (c) Candidates are filtered by field-of-view inclusion, obstacle clearance and visibility. (d) Per-azimuth representatives are selected by visible sample count and terminal cost, then reduced to a diverse set within the viewpoint budget. The scene and display dimensions are illustrative.

Initialization requires $\text{pred}(i) \subseteq \mathcal{R}_t$. A transition requires $j \notin S$, $\text{pred}(j) \subseteq \mathcal{R}_t \cup S$, and a finite path cost. Unavailable states have infinite value. Backtracking from $\min_{i,k} D(\mathcal{W}_t, i, k)$ yields the visit order and observation poses.

For m admitted tasks and at most K admitted candidates per task, the recurrence has $O(2^m m^2 K^2)$ transition complexity, excluding geometric path search. The recurrence yields the minimum-cost sequence over the admitted task window and pose sets. The robot executes the first visit and then replans using the updated observations. During local recovery, the remaining views of the current location are prioritized to avoid repeatedly leaving an unresolved search region.

D. Execution, Recovery, and Semantic Updates

The execution loop associates each visit with an observation and a task-policy decision. Figure 8 illustrates how local reinspection and semantic recovery preserve the cushion-search objective within a multi-object inspection mission.

1) Trajectory generation and observation: The selected geometric route is converted into a smooth, collision-checked trajectory toward the next observation pose. Habitat execution uses cubic B-spline smoothing, while physical flight uses MINCO [10] within safe corridors, as illustrated in Fig. 7(b). Trajectory optimization refines continuous motion after the discrete task–viewpoint decision. At the selected pose, the robot acquires an RGB-D observation and evaluates the task's verification target.

Text-prompted SAM 3.1 provides category detections for online verification. Depth and camera pose project detections into the global frame, where spatial association checks their consistency with mapped objects or accumulated novel-instance tracks. A positive visual outcome requires both a target-category detection and geometric confirmation. Spatial relations determine where to search, while the visual check establishes category presence and geometric consistency; relation-specific truth is not a separate termination criterion. The resulting observation updates the task state and determines the next planning action.

2) Three levels of negative evidence: A *pose-level miss* leaves the task unresolved while alternative local views remain. These views provide different observation directions around the same anchor. A *location-level exhaustion* occurs when the local candidate set is exhausted or the local search limit is reached. For a locate-target request that permits autonomous recovery, the missing target, instruction, failed anchors, and scene inventory condition alternative room–anchor–region hypotheses. Grounding these hypotheses in mapped entities yields new search regions and feasible camera poses. Search continues within a bounded set of locations, with room-level alternatives available when anchor-specific views are exhausted.

A *terminal task outcome* is emitted only when the target is found or no permitted continuation remains. A terminal not-found indicates that the permitted search locations and viewpoints have been exhausted without confirming the target. The task intent determines whether it is an acceptable presence-check result, an unresolved locate-target task, or the trigger for an explicit fallback. Only then does CPTG evaluate its conditional rules.

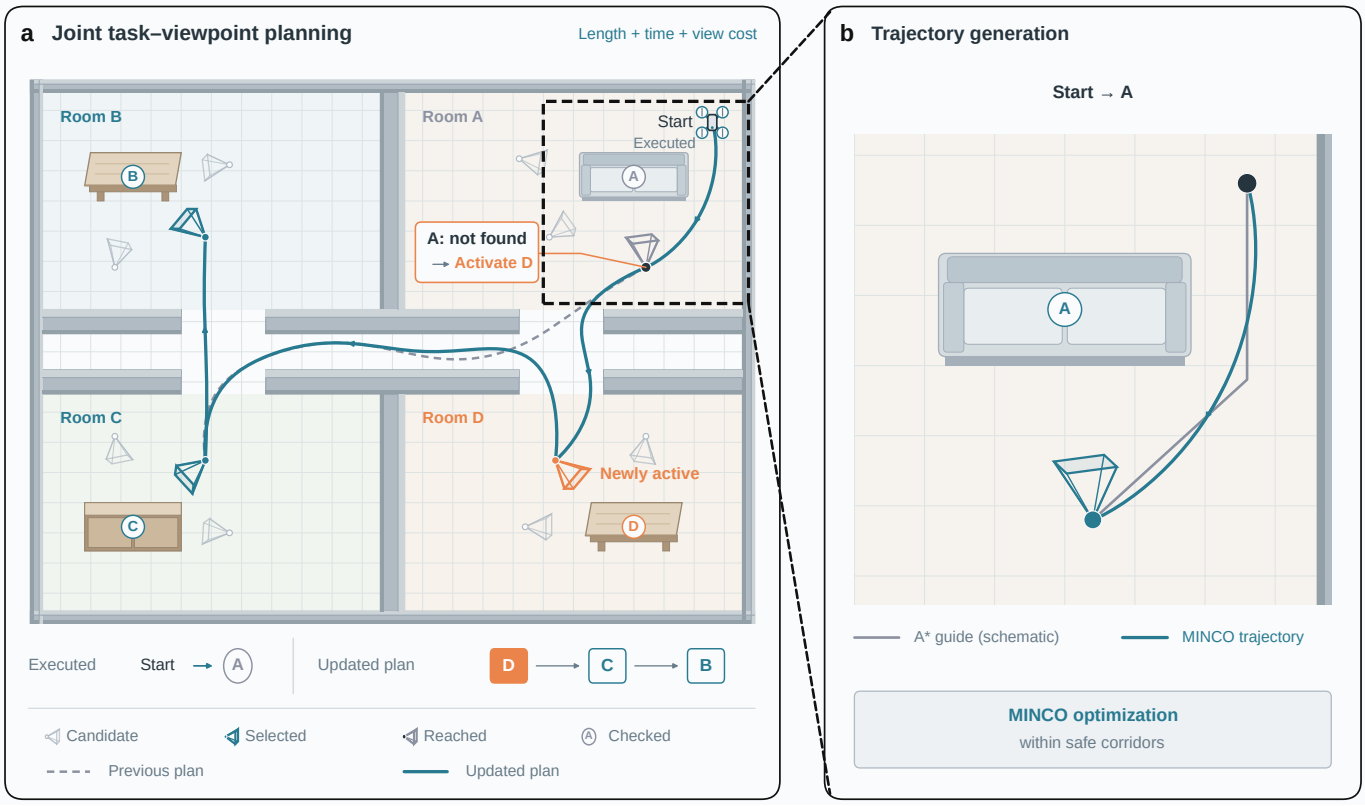

Fig. 7. Joint task–viewpoint planning and trajectory generation. (a) Each room has one illustrative task anchor and several candidate views. After the Start → A visit, a terminal not-found outcome activates D and changes the remaining order from C → B to D → C → B. Grey dashed curves show the previous plan; teal shows the updated route and executed prefix. Faint and bold wireframes indicate candidate and selected views, and orange marks the newly activated task. (b) The dashed box identifies the enlarged region: an A* guide (grey, simplified for display) supports safe-corridor construction and MINCO optimization (teal). This is a mechanism illustration with a computed trajectory.

3) Coverage-aware search confidence: For a semantic search region r, let $b_r \in [0,1]$ denote its search confidence, $\mathcal{P}_r$ its spatial samples, U_r the samples already covered, and $V(q)$ the samples with an unobstructed ray from pose q. The ratio $Q_r(q) = |V(q)|/\max(1,|\mathcal{P}_r|)$ measures line-of-sight coverage. It is a geometric search proxy; camera FoV is enforced during candidate generation through Eq. 3. A negative observation gives

$$\begin{aligned} \Delta Q_r(q) &= \frac{|V(q)\setminus U_r|}{\max(1,|\mathcal{P}_r|)}, \\ b_r^+ &= b_r[1-\rho\,\Delta Q_r(q)Q_r(q)], \\ U_r^+ &= U_r \cup V(q), \end{aligned} \tag{8}$$

where $\rho \in [0,1]$ controls the strength of negative evidence. A positive observation sets $b_r^+ = 1$. Because confidence decreases only with newly covered samples, repeated observations of the same region do not accumulate redundant negative evidence. The bounded confidence score prioritizes subsequent search regions. Semantic-region candidates are ranked by

$$C_r^{\text{search}}(q) = \lambda_L \bar{L}_r^{\text{cover}} + \lambda_Q[1-Q_r(q)] + \lambda_b(1-b_r), \tag{9}$$

where $\bar{L}_r^{\text{cover}}$ is the normalized region-coverage travel estimate and the nonnegative weights balance travel, coverage, and semantic confidence. Initial confidence reflects the relevance of each recovery hypothesis. This cost replaces the ordinary terminal cost for semantic-region candidates, directing subsequent observations toward accessible regions that remain plausible.

4) Object confidence and new instances: When the mapped object is also the verification target, its confidence increases toward one after a positive observation and decreases after a negative observation. These updates are $p^+ = p + \alpha_+(1-p)$ and $p^+ = \alpha_- p$, with $\alpha_+, \alpha_- \in (0,1)$. Failure to find a different target near a surrogate anchor does not reduce the anchor's existence confidence. Unmatched same-category RGB-D detections are associated across observations, and consistently supported tracks are added as new instances after execution. Regional confidence supports the ongoing search, while the revised object map supplies priors for later missions.

The complete loop thus grounds the instruction, constructs and filters views, selects a task–pose visit, executes it, and updates the appropriate evidence level. It continues local search when warranted and returns to global scheduling after task resolution or a change of search location. This preserves task logic while allowing geometric choices to respond to observations.

V. Evaluation

We examine navigation and visual observation in Habitat and physical UAV experiments. The simulation examples cover

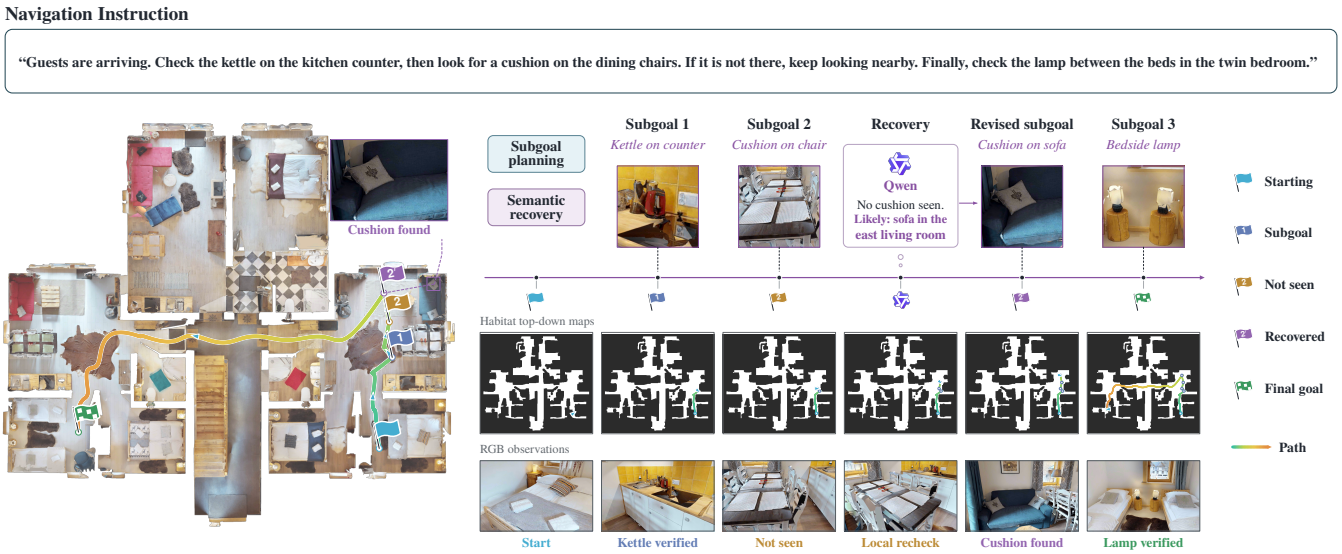

Fig. 8. Illustration of semantic recovery within a multi-object inspection task. The sequence checks a kettle on the kitchen counter, searches the dining chairs for a cushion, and checks a bedside lamp. After local reinspection yields no cushion observation, the recovery module proposes the sofa in the eastern living room as an alternative search anchor. The cushion is then located, and the mission continues to the bedroom. Flags associate key observation poses with the trajectory, top-down maps, and RGB views.

reorderable checks, recovery after unsuccessful observations, and explicit conditional branches.

A. Experimental Setup

Simulation uses Habitat [20] with HM3D indoor scenes [21]. Each episode combines a language-specified task, a preconstructed semantic–geometric map, and RGB-D observations acquired during navigation. The scenes contain multiple rooms and, where applicable, multiple floors. The qualitative examples examine how the robot coordinates observations, responds to an unsuccessful search, and follows conditional task requirements.

B. Benchmark and Metrics

The task families include single-target navigation, ordered and reorderable multi-target checks, conditional alternatives, and recovery when an expected target is missing. Task success requires the requested terminal outcomes while respecting precedence and activated branches. Reaching an anchor alone does not satisfy a visual check. A bounded negative outcome can resolve a presence check, whereas a locate-target request remains unsuccessful without discovery or its prescribed fallback.

We define trajectory length (TL) from the executed 3D path and full-task success rate (SR) over all attempted episodes. For episodes with a valid branch- and order-consistent reference route, success weighted by path length is

$$\mathrm{SPL} = \frac{1}{|\mathcal{D}_g|} \sum_{e \in \mathcal{D}_g} S_e \frac{L_e^{\mathrm{ref}}}{\max(L_e^{\mathrm{ref}}, L_e)}, \tag{10}$$

where L_e and L_e^{ref} denote the executed and reference route lengths, respectively. The set $\mathcal{D}_g$ contains episodes with a defined geometric goal and a valid reference route, and $S_e \in \{0, 1\}$ denotes full-task success. Reference routes obey the same precedence and activated-branch constraints as the evaluated mission. Absence checks without a geometric goal are assessed by their required terminal outcome and excluded from SPL. Execution time and task-level outcomes complement path efficiency.

C. Qualitative Execution in Habitat

Figure 9 illustrates execution across different indoor scenes. For multi-target instructions, the robot visits observation locations across rooms and floors. When an expected object is not observed, it retries alternative local views before extending the search to another room, retaining the original search objective. For conditional instructions, a successful observation activates the subsequent task specified by the instruction. These behaviors illustrate how task logic and visual feedback jointly guide mission progress, while distinguishing recovery within an ongoing task from activation of a new task.

D. Comparison with Existing Methods

Results to be added.

E. Ablation Studies

Results to be added.

F. Physical UAV Experiments

The deployment platform comprises a quadrotor, a Jetson Orin NX, a PX4 flight controller, a Livox Mid-360 LiDAR, and an Intel RealSense D435 RGB-D camera. LiDAR–inertial localization uses the FAST-LIO family [22] and aligns the vehicle with the mapped environment.

The physical experiments cover cross-room flight, online target recognition, conditional task activation, local viewpoint

Fig. 9. Simulation visualizations. Each row shows a navigation episode, with the language instruction above representative first-person UAV keyframes along the trajectory.

retries, and recovery search across rooms. During execution, visual observations update the task state: an unsuccessful local observation leads to another viewpoint, while exhaustion of the local search prompts exploration of an alternative room. For instructions with conditional requirements, the observed outcome determines whether a subsequent task becomes active. These experiments exercise the connection between task logic, viewpoint selection, perception, and physical motion.

Figure 1 presents a multi-room execution example with the UAV route and camera observations. The reconstruction provides spatial context, while the observation sequence shows the views acquired along the flight.

VI. Conclusion

LogicFly addresses long-horizon language-guided aerial navigation in environments where a reusable semantic map can be constructed before task execution. Its CPTG distinguishes precedence and explicit conditional alternatives, while JTVP couples task ordering with three-dimensional observation-pose selection. A staged recovery loop interprets negative observations at the pose, location, and task levels and updates search confidence from newly observed coverage. Together, these mechanisms connect semantic priors to observable goals and evidence-driven task progress.

The Habitat examples illustrate multi-room checks, task-preserving recovery, and outcome-triggered activation. Physical UAV experiments demonstrate the corresponding closed-loop behaviors, including online recognition, conditional execution, local retries, and cross-room recovery. Map uncertainty and recognition errors can affect observation quality, while the rolling horizon balances planning cost and the range of task coordination. Future work will investigate robustness to changes in scene geometry and object placement, and improve long-horizon planning efficiency.

References

[1] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *IEEE International Conference on Robotics and Automation*, 2023, pp. 10 608–10 615.

[2] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, "Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation," in *Robotics: Science and Systems*, 2024.

[3] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, and A. van den Hengel, "Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 3674–3683.

[4] J. Krantz, E. Wijmans, A. Majumdar, D. Batra, and S. Lee, "Beyond the nav-graph: Vision-and-language navigation in continuous environments," in *European Conference on Computer Vision*, 2020, pp. 104–120.

[5] X. Song, W. Chen, Y. Liu, W. Chen, G. Li, and L. Lin, "Towards long-horizon vision-language navigation: Platform, benchmark and method," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025, pp. 12 078–12 088.

[6] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Sünderhauf, "SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning," in *Proceedings of the Conference on Robot Learning*, vol. 229, 2023, pp. 23–72.

[7] Z. Dai, A. Asgharivaskasi, T. Duong, S. Lin, M.-E. Tzes, G. J. Pappas, and N. Atanasov, "Optimal scene graph planning with large language model guidance," in *IEEE International Conference on Robotics and Automation*, 2024, pp. 14 062–14 069.

[8] J. Krantz, S. Banerjee, W. Zhu, J. Corso, P. Anderson, S. Lee, and J. Thomason, "Iterative vision-and-language navigation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 921–14 930.

[9] Q. Gu, A. Kuwajerwala, S. Morin, K. M. Jatavallabhula, B. Sen, A. Agarwal, C. Rivera, W. Paul, K. Ellis, R. Chellappa, C. Gan, C. M. de Melo, J. B. Tenenbaum, A. Torralba, F. Shkurti, and L. Paull, "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," in *IEEE International Conference on Robotics and Automation*, 2024, pp. 5021–5028.

[10] Z. Wang, X. Zhou, C. Xu, and F. Gao, "Geometrically constrained trajectory optimization for multicopters," *IEEE Transactions on Robotics*, vol. 38, no. 5, pp. 3259–3278, 2022.

[11] X. Chen, Z. Liu, J. Ma, B. Du, T. Zhang, X. Wang, and B. Zhou, "AirHunt: Bridging VLM semantics and continuous planning for efficient aerial object navigation," *arXiv preprint arXiv:2601.12742*, 2026.
[12] G. Zhou, Y. Hong, and Q. Wu, "NavGPT: Explicit reasoning in vision-and-language navigation with large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 7, 2024, pp. 7641–7649.
[13] J. Chen, B. Lin, R. Xu, Z. Chai, X. Liang, and K.-Y. K. Wong, "MapGPT: Map-guided prompting with adaptive path planning for vision-and-language navigation," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024, pp. 9796–9810.
[14] J. Zhang, Z. Li, S. Wang, X. Shi, Z. Wei, and Q. Wu, "SpatialNav: Leveraging spatial scene graphs for zero-shot vision-and-language navigation," *arXiv preprint arXiv:2601.06806*, 2026.
[15] N. P. Bhatt, Y. Yang, R. Siva, P. Samineni, D. Milan, Z. Wang, and U. Topcu, "VLN-Zero: Rapid exploration and cache-enabled neurosymbolic vision-language planning for zero-shot transfer in robot navigation," *arXiv preprint arXiv:2509.18592*, 2025.
[16] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205, 2023, pp. 1769–1782.
[17] Y. Zhang, X. Chen, C. Feng, B. Zhou, and S. Shen, "FALCON: Fast autonomous aerial exploration using coverage path guidance," *IEEE Transactions on Robotics*, vol. 41, pp. 1365–1385, 2024.
[18] N. Carion, L. Gustafson, Y.-T. Hu *et al.*, "SAM 3: Segment anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.
[19] D. DeTone, T. Shen, F. Zhang, L. Ma, J. Straub, R. Newcombe, and J. Engel, "Boxer: Robust lifting of open-world 2D bounding boxes to 3D," *arXiv preprint arXiv:2604.05212*, 2026.
[20] M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra, "Habitat: A platform for embodied AI research," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 9339–9347.
[21] S. K. Ramakrishnan, A. Gokaslan, E. Wijmans, O. Maksymets, A. Clegg, J. Turner, E. Undersander, W. Galuba, A. Westbury, A. X. Chang, M. Savva, Y. Zhao, and D. Batra, "Habitat-Matterport 3D Dataset (HM3D): 1000 large-scale 3D environments for embodied AI," in *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.
[22] W. Xu and F. Zhang, "FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, 2021.